Case 02 · Architecture: Private Exposure Registry

Chapter 3 (Architecture) of the Magshimim project-book template: overview, mapping to the spec, and data design and information entities. Three sections not in the template were added for the comparison stage: architectural patterns and trade-offs, failure scenarios and build order, and a closure check.

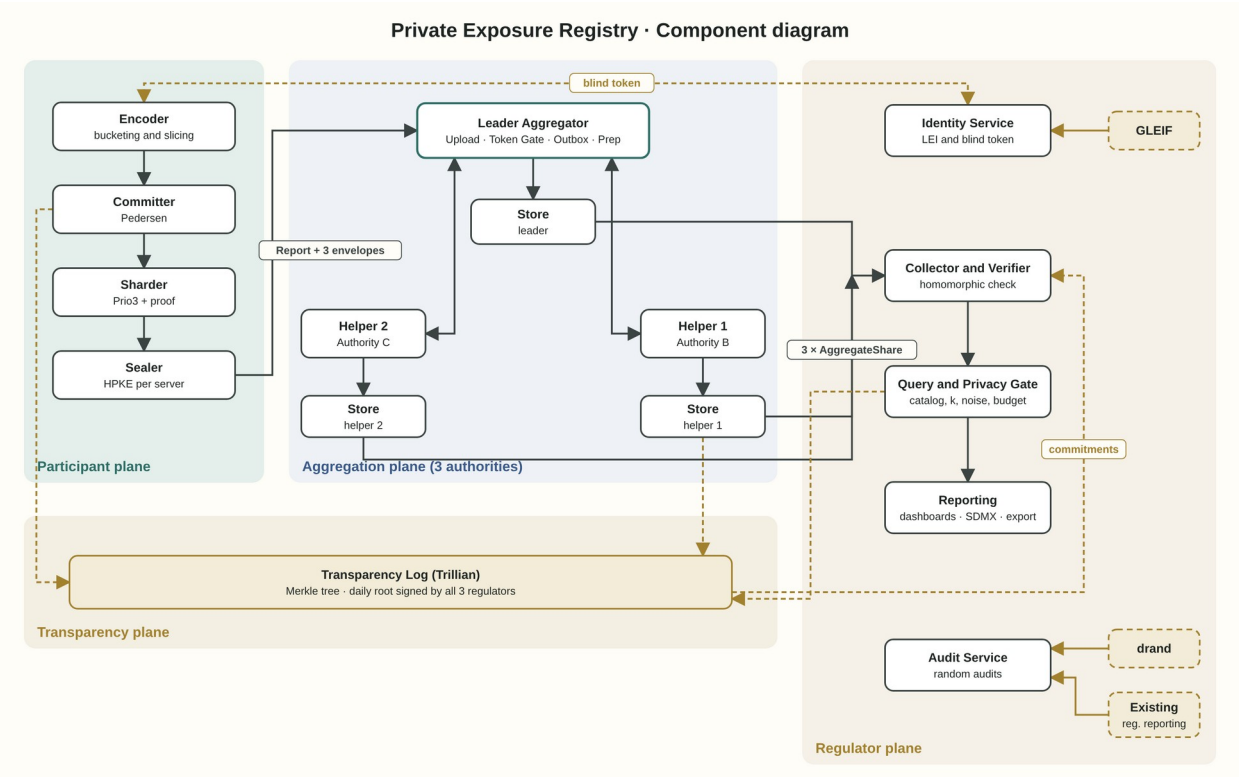
Overview

The principle that organises the whole structure

In this system no one trusts anyone, and the structure reflects that. There are four planes, and every component belongs to exactly one of them. Information passes between planes only in a form the receiving plane cannot exploit:

- **The participant plane:** the application running at the reporting entity. Raw data never leave it, only encrypted envelopes and commitments.
- **The aggregation plane:** three servers, each at a different supervisory authority. Each sees only its own share, which looks like noise.
- **The regulator plane:** identity, result verification, the query catalog, and publication. Sees sums, never an individual report.
- **The transparency plane:** a public log anyone can read and no one can change.

Component diagram



The components

- **Participant Client (participant plane):** a Rust application, both desktop and WebAssembly in the browser. Four sub-components in order of operation: Encoder (turns the loan book into a bucketed, clipped vector), Committer (computes Pedersen commitments), Sharder (Prio3: splitting and validity proof), and Sealer (HPKE: an encrypted envelope per server).

- **Identity Service (regulator plane):** verifies an LEI against the GLEIF database and the list of supervised entities, and issues one blind participation token per entity per period. The signing key lives in an HSM.
- **Leader Aggregator (aggregation plane):** the entry point for reports. Four sub-components: Upload API (accepts reports and returns a receipt), Token Gate (checks the token's signature and reuse), Forward Outbox (forwards the helpers' envelopes to them, durably), and Prep Coordinator (runs the joint verification stage).
- **Helper Aggregator 1 and 2 (aggregation plane):** each opens its own envelope, takes part in verification, and aggregates. The only difference from the leader: they do not receive reports directly from participants.
- **Aggregate Store (one per server):** a PostgreSQL holding the shares and partial sums of that server only.
- **Collector and Verifier (regulator plane):** receives the three partial sums, adds them, and checks them against the sum of commitments. If the check fails, it runs the binary search.
- **Query and Privacy Gate (regulator plane):** the query catalog, the K=5 threshold gate, Discrete Gaussian noise for the public release, and the privacy-budget ledger.
- **Reporting (regulator plane):** dashboards for regulators, the public SDMX release, and the distribution export.
- **Audit Service (regulator plane):** selecting participants for random audit by a drand value, and comparing opened commitments with existing regulatory reporting.
- **Transparency Log (transparency plane):** Trillian, a Merkle-tree log. Every component writes to it, and everyone can read from it.
- **External components:** the GLEIF database (identity), the drand network (public randomness), and existing regulatory reporting (Form PF, AIFMD Annex IV) that the regulator already holds.

Component table: role and interfaces

Component	Role	Incoming interfaces	Outgoing interfaces
Participant Client	Encoding, commitment, splitting, encryption	Local loan book, template, HPKE keys, token	Report and CommitmentSet to the Leader
Identity Service	LEI verification and blind-token issuance	Joining request, blinded token	Blind signature on the token
Leader Aggregator	Intake, token check, forwarding, verification coordination	Reports from participants	Envelopes to helpers, receipts, AggregateShare
Helper Aggregator 1, 2	Opening the envelope, verification, aggregation	Envelopes and verification messages from the Leader	Verification messages, AggregateShare
Aggregate Store	Storing one server's shares and partial sums	Writes from its own server only	Reads from its own server only
Collector and Verifier	Adding sums and homomorphic check	Three AggregateShares, commitments from the log	A verified sum or a failure identified
Query and Privacy Gate	Catalog, threshold, noise, budget	Verified sum, counts, a query request	Exact answer for the regulator, noised answer for the public

Component	Role	Incoming interfaces	Outgoing interfaces
Reporting	Dashboards, release, export	Answers from the Gate	Dashboards, SDMX, Parquet
Audit Service	Random audits	drand value, log, existing regulatory reports	Audit results to the log
Transparency Log	Immutable log	Events from every component	Signed root, inclusion and consistency proofs

Mapping to the spec

Spec feature	Components involved	Order of operations and why
1. Onboarding and identity	Participant Client → Identity Service → GLEIF	The client blinds the token before sending it for signature, so the regulator signs without seeing it and cannot link it to a report
2. Local encoding	Participant Client (Encoder)	Happens before anything else, on the participant's machine only
3. Commitments and receipts	Participant Client (Committer) → Leader (Upload API) → Transparency Log	The commitment is created before splitting, so what was sent can be proven before the servers touch it
4. Splitting and proof	Participant Client (Sharder, Sealer)	The proof is split together with the data, so no server holds a whole proof
5. Aggregation	Leader → Helpers → Aggregate Store on each server	Token check before anything, joint verification before aggregation, and only reports that pass are aggregated
6. Correctness check	Aggregate Stores → Collector and Verifier → Transparency Log	The homomorphic check runs before the sum reaches the catalog. A sum that has not passed is not opened to any question
7. Catalog and privacy	Collector and Verifier → Query and Privacy Gate	The threshold is checked before noise, and the budget before release. A question that would exceed the budget is refused before it is computed
8. Transport and operations	Sealer, Forward Outbox, mTLS between servers	A helper's envelope is held at the leader until it returns. The leader cannot open it
9. Log and audit	Every component → Transparency Log; Audit Service → drand	Every component writes to the log before confirming an action, so there is no action without a record
10. Reports and release	Query and Privacy Gate → Reporting	The regulator dashboard shows exact values, the public release noised values with a margin of error. Both point to the verification record in the log

Data design and information entities

Information passed between components (not stored in this form)

- BlindedToken and BlindSignature, between Participant Client and Identity Service: a blinded token and the signature on it. After unblinding, the client holds a token the regulator has never seen.
- **Report, from Participant Client to the Leader:** report identifier, task_id, public share, three HPKE envelopes (one per server), and the token.
- CommitmentSet, from Participant Client to the Leader and from there to the log: a commitment per cell.
- **PrepMessage, between the servers through the Leader:** Prio3's verification messages. Short, and they reveal no values.
- **AggregateShare, from each server to the Collector:** a partial sum, a partial sum of the r values, a report count, and a fingerprint of the report list.
- **QueryRequest and QueryResponse, between Reporting and the Gate:** a catalog question identifier, and an answer with blocked cells marked.

Information that is stored

- **In each Aggregate Store:** tasks, reports (that server's envelope only, until the end of the retention period), aggregate_shares, and at the leader also used_tokens and pending_forwards.
- **At the regulator:** participants, token_issuance, templates, catalog_versions, batches, budget_ledger, releases, audit_selections.
- **In the transparency log:** every event as a leaf in the tree, and a daily root signed by all three regulators.
- **At the participant:** the r values and clipping results, locally, for future audit. Never on the servers.

Deliberate separation

There is no single place in the system where a participant's identity and the content of its report are together. Identity (LEI) lives only in the Identity Service. Content lives only split across three servers. The blind token is the bridge, and it is built so that it cannot be crossed in the opposite direction.

Architectural patterns and trade-offs

The log as the source of truth (Event Sourcing)

In most systems the database is the truth and the log is a record. Here it is the reverse: the transparency log is the source of truth, and the state at the regulator (which batches were verified, how much budget was spent) is a "view" built by reading the log. Every component writes an event to the log before confirming an action.

- **What we gain:** the regulator's whole state can be rebuilt from the log alone. Any claim such as "0.6 of the budget has been spent" can be checked by anyone. And history cannot be changed quietly.
- **What we pay:** writing to the log is part of every action, so the log must be available. And changing an event's structure after it is written is complicated, because old ones are never deleted.

Separating writes and reads (CQRS)

The write path (receiving reports, verification, aggregation) and the read path (questions, dashboards, release) are two completely separate models. The write path works on encrypted shares across three servers. The read path works only on verified sums at the regulator.

- **What we gain:** there is no path along which a regulator's question touches an individual report. The separation is not just code organisation; it is part of security.
- **What we pay:** an answer is available only after a batch has closed and been verified. There is no "live question" on reports still in progress.

Transactional Outbox

When the leader receives a report, it stores the report and the "forward messages" for both helpers in the same database transaction. A separate process sends the messages and deletes them only after acknowledgement.

- **What we gain:** there is no state in which a report was accepted but a helper did not receive its share. This is what makes the "all three servers needed" decision possible without losing reports when a helper is unavailable.
- **What we pay:** forwarding to a helper is "eventual", not immediate. Duplicates are possible, so a helper must recognise a report it already has by report_id.

The three-key rule (Multi-Party Governance)

A change to the query catalog, the reporting template or the budget requires the approval of all three regulators, each signing with its own key, and the change is recorded in the log. Similar to the "two-person rule" for vaults and weapons systems, but with three.

- **What we gain:** no single authority can add a question designed to expose a participant, or quietly increase the budget.
- **What we pay:** slow changes. If one authority does not respond, nothing changes. That is deliberate.

Building on a standard rather than inventing

The system's core (splitting, proof, aggregation) is Prio3 and DAP per the IETF drafts, with the libprio-rs and Janus implementations. Our code adds what the standard does not provide: three servers, commitments and homomorphic verification, blind tokens, the catalog and the budget.

- **What we gain:** cryptography reviewed by others and running in production.
- **What we pay:** the drafts are still changing. Extending to three servers goes beyond what DAP defines today, so it is our most sensitive code.

Failure scenarios

What fails	What happens	Why that is acceptable (or not)
One helper is unavailable	Reports keep arriving. Its envelopes wait in the Outbox. The batch does not close until it returns	The trade-off we chose: a delayed report, not a leak and not a loss
The leader is unavailable	Participants cannot report. The application keeps the report encrypted and retries	No data are lost, and the participant gets a receipt when submission succeeds
The commitment check fails	The sum is not opened. Binary search finds the faulty report or server	No answer is better than a wrong answer
A server sends a tampered share	The homomorphic check fails, and the binary search points to the server	The server is identified, with proof in the log
A report fails the validity proof	The report is dropped, and the receipt shows that it was dropped and why	A participant can check itself and correct
The privacy budget runs out	No more public releases for the period. Regulators still see exact values	We do not sell privacy for one more chart
The log is unavailable	Every action that needs a record stops	No action without a witness. That is deliberate, and it is why the log is replicated

Build order

- **Sprint 1, the core with two servers:** Participant Client (Encoder, Sharder), a Leader and one Helper, on libprio-rs and Janus as they are. Goal: reporting, verification and aggregation end to end according to the standard.
- **Sprint 2, three servers:** extending to a second helper, Forward Outbox, and HPKE per server.
- **Sprint 3, commitments and verification:** Committer, Collector and Verifier, homomorphic check and binary search.
- **Sprint 4, identity:** Identity Service, blind tokens, Token Gate, and the GLEIF connection.
- **Sprint 5, transparency:** Transparency Log (Trillian), writes from every component, signed root.
- **Sprint 6, catalog and privacy:** Query and Privacy Gate, the differencing check, OpenDP and the budget.
- **Sprint 7, reports and audit:** Reporting, SDMX release, Audit Service with drand.

Closure check

The closure check asks a simple question: can the system keep its central promise end to end, using only components inside its own boundaries? The project's central promise, from ideation: to see a crisis that arrives slowly, before it arrives fast. In other words, to say something about the market as it is today.

Component or input	Inside the boundaries?	Does the promise depend on it?
The market's exposure and leverage data	Yes: collected from the participants themselves, through the system	Yes, and it is the decisive input
Encoding, splitting, aggregation and verification	Yes	Yes, and fully under control
Catalog, threshold and privacy	Yes	Yes, and fully under control
The GLEIF database	No	No: identity only. Replaceable by the list of entities the regulator already holds
The drand network	No	No: audit selection only. Replaceable by any public randomness source
Existing regulatory reporting	No, but already at the regulator	Only for random audits, not for the result

The decisive input, what each entity holds and at what leverage, enters the system through its own components, passes through it, and leaves it as a verified picture. The three external dependencies serve identity and audit, not the answer, and each of them is replaceable.

What the system does not promise, and the spec says so too: it shows the picture, not what will happen to it. It can say "much of the exposure carries rates above 5%, and a third of funds are leveraged above 4 times". It cannot say "and if rates rise another point, this is what will happen". But what it does promise, it promises end to end, within its own boundaries.